



university of
 groningen

Languages and Machines

L9: Variations of Turing machines

Jorge A. Pérez

Bernoulli Institute for Math, Computer Science, and AI
University of Groningen, Groningen, the Netherlands



Regular $\leftrightarrow$ Finite State Machines (FSMs)

Context-free $\leftrightarrow$ Pushdown Machines

Context-sensitive $\leftrightarrow$ Linearly-bounded Machines

Decidable $\leftrightarrow$ Always-terminating Turing Machines

Semi-decidable $\leftrightarrow$ Turing Machines

Outline



From Last Lecture

Variations of TMs

- Multitrack TMs

- The Example Revisited (I)

- Multitape TMs

- The Example Revisited (II)

- Simulating Multitape with Multitrack

Non-Deterministic TMs (NTMs)

- Adding Non-Determinism

- Complexity Classes

Closure Properties

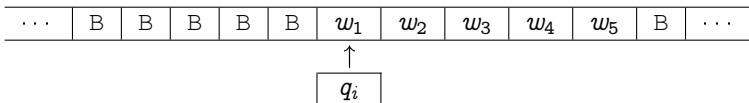
Turing Machines (TMs)



A (simple) **Turing machine** M includes

- A set of **states** Q , with **start state** $q_0 \in Q$
- The **tape alphabet** Γ is such that $\Gamma \cap Q = \emptyset$.
There is a **blank symbol** $B \in \Gamma \setminus \Sigma$
- The **input alphabet** Σ is such that $\Sigma \subseteq \Gamma \setminus \{B\}$

Graphically:



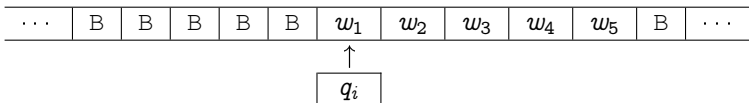
Turing Machines (TMs)



A (simple) **Turing machine** M includes

- A set of **states** Q , with **start state** $q_0 \in Q$
- The **tape alphabet** Γ is such that $\Gamma \cap Q = \emptyset$.
There is a **blank symbol** $B \in \Gamma \setminus \Sigma$
- The **input alphabet** Σ is such that $\Sigma \subseteq \Gamma \setminus \{B\}$

Graphically:



A transition:

- ▶ changes the state
- ▶ writes a symbol on the square scanned by the head
- ▶ moves the head one square to the left (L) or to the right (R)

Turing Machines (TMs)



A (simple) **Turing machine** M is a quintuple $(Q, \Sigma, \Gamma, \delta, q_0)$ where

- Q is a set of **states**
- $q_0 \in Q$ is the **start state**
- Γ is the **tape alphabet**, a set of symbols disjoint from Q .
Contains a **blank symbol** B , not in Σ
- $\Sigma \subseteq \Gamma \setminus \{B\}$ is the **input alphabet**
- The transition function δ is a *partial* function such that

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

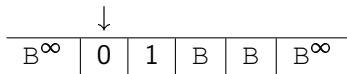
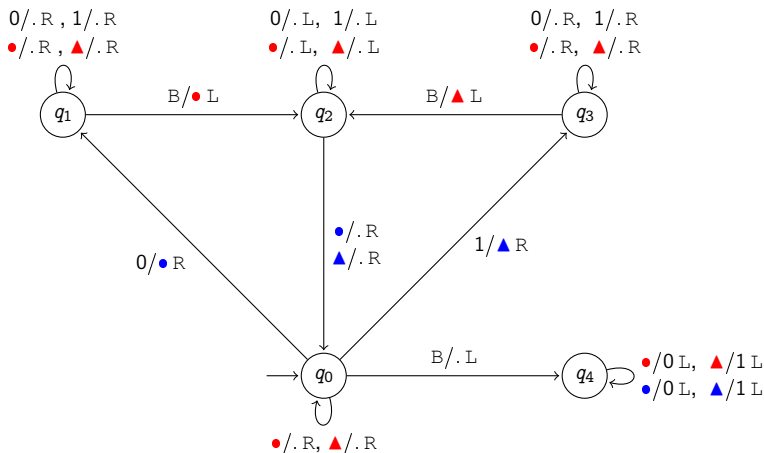
Note:

- ▶ Accepting states $F \subseteq Q$ useful but not indispensable for defining acceptance
- ▶ From δ we derive a transition relation $\vdash$ on **configurations** (strings in $\Gamma^* Q \Gamma^*$)

From Last Lecture: Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

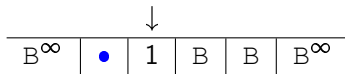
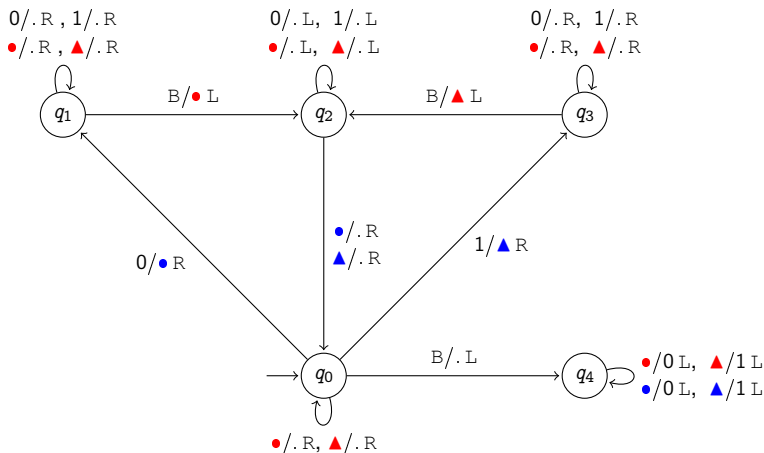


(State = q_0)

From Last Lecture: Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

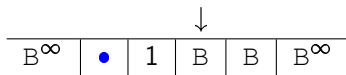
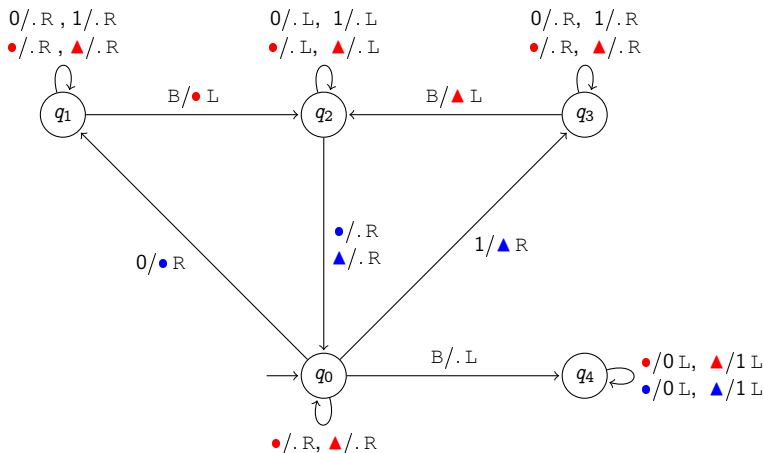


(State = q_1)

From Last Lecture: Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

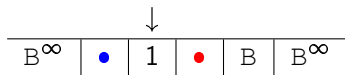
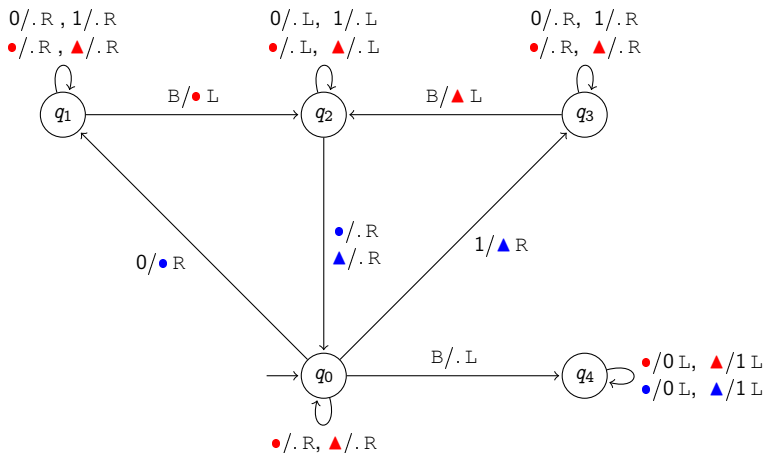


(State = q_1)

From Last Lecture: Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

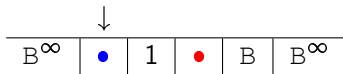
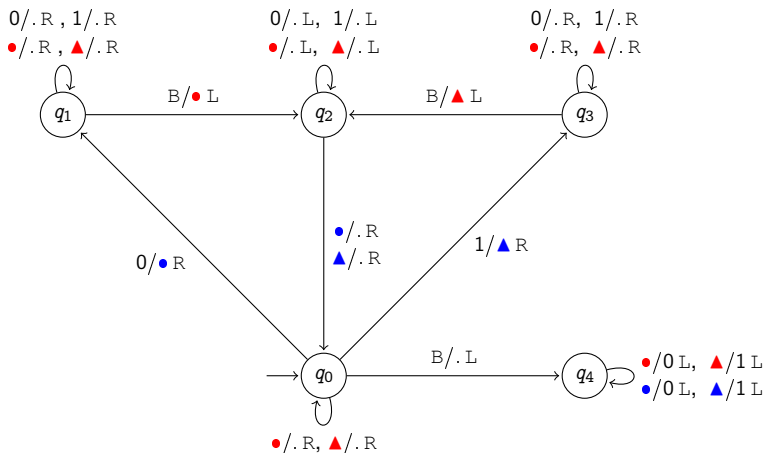


(State = q_2)

From Last Lecture: Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

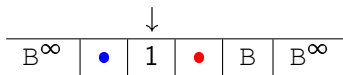
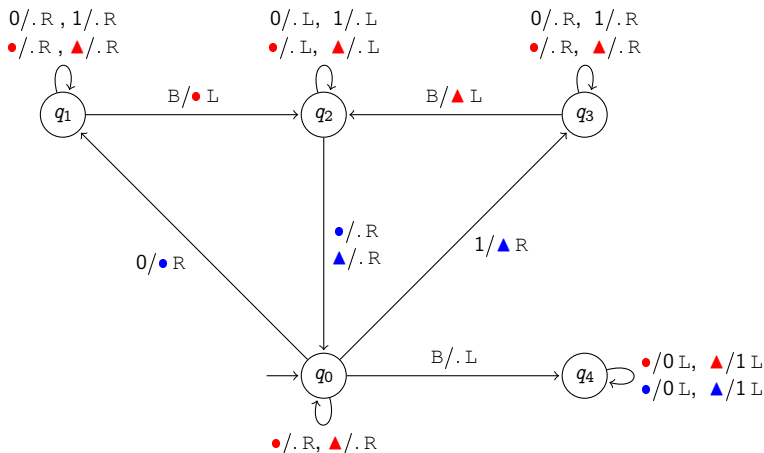


(State = q_2)

From Last Lecture: Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

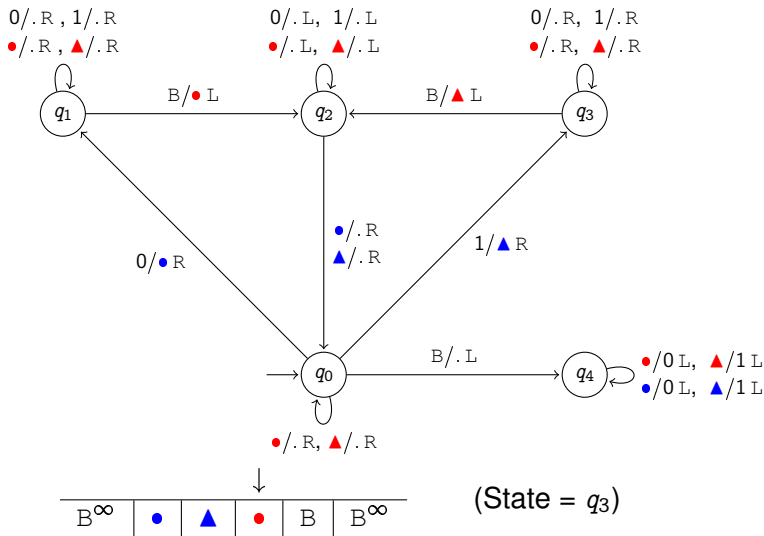


(State = q_0)

From Last Lecture: Example 2



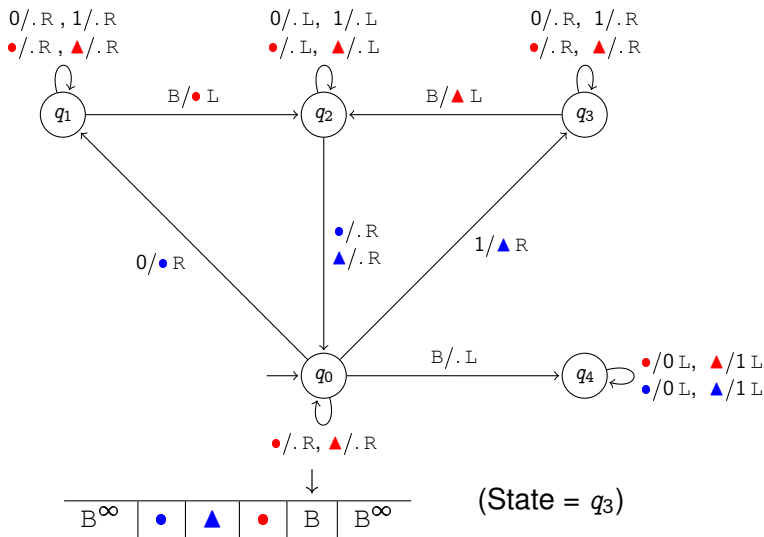
A TM that duplicates the input string $w \in \{0, 1\}^*$.



From Last Lecture: Example 2



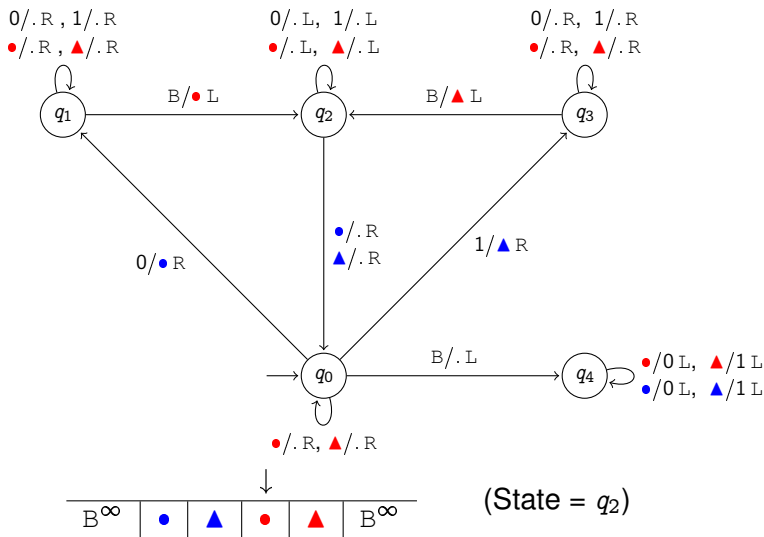
A TM that duplicates the input string $w \in \{0, 1\}^*$.



From Last Lecture: Example 2



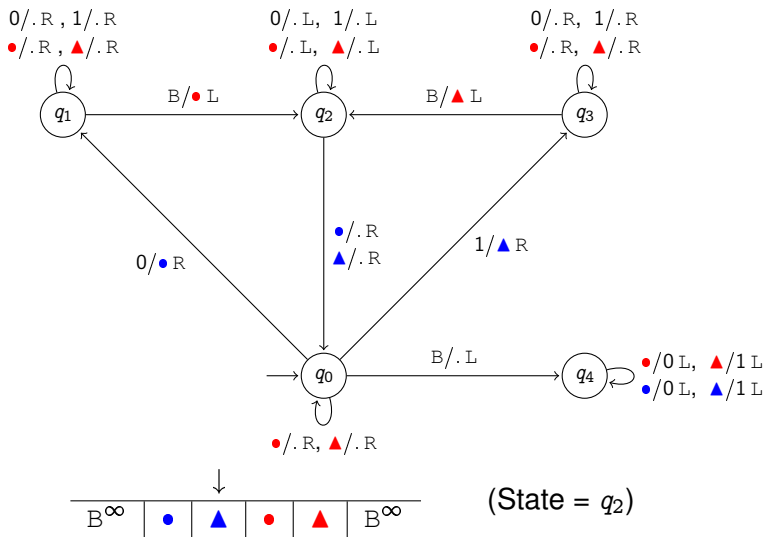
A TM that duplicates the input string $w \in \{0, 1\}^*$.



From Last Lecture: Example 2



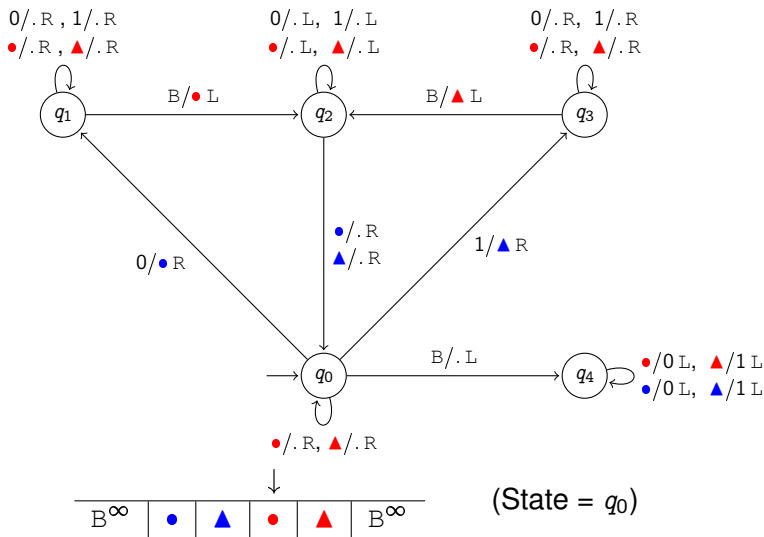
A TM that duplicates the input string $w \in \{0, 1\}^*$.



From Last Lecture: Example 2



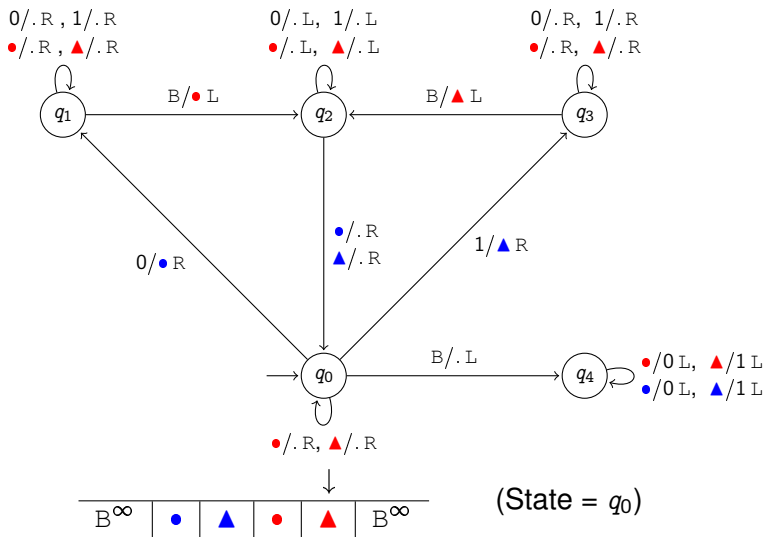
A TM that duplicates the input string $w \in \{0, 1\}^*$.



From Last Lecture: Example 2



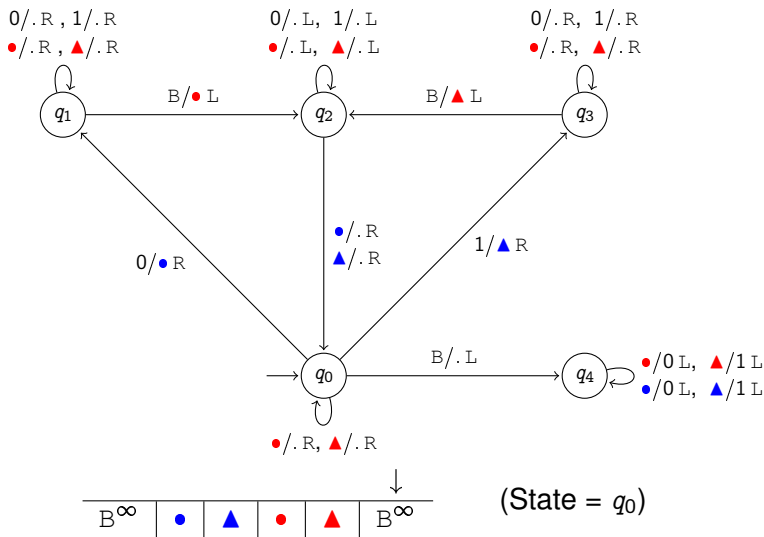
A TM that duplicates the input string $w \in \{0, 1\}^*$.



From Last Lecture: Example 2



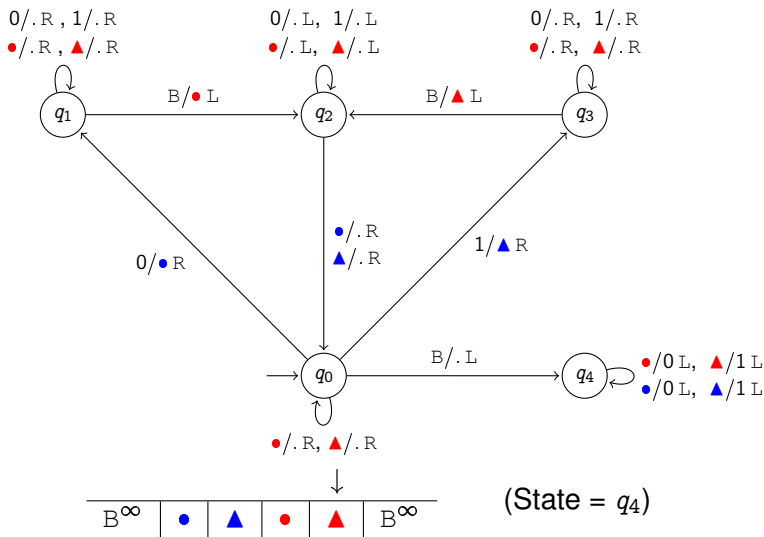
A TM that duplicates the input string $w \in \{0, 1\}^*$.



From Last Lecture: Example 2



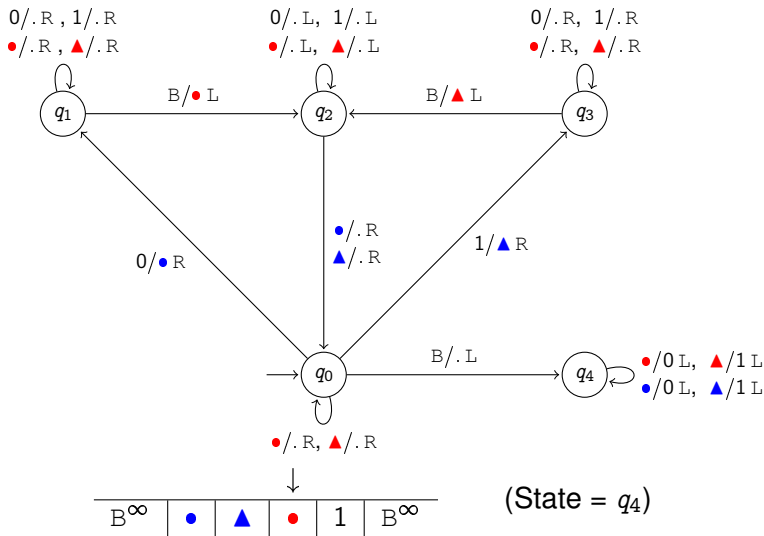
A TM that duplicates the input string $w \in \{0, 1\}^*$.



From Last Lecture: Example 2



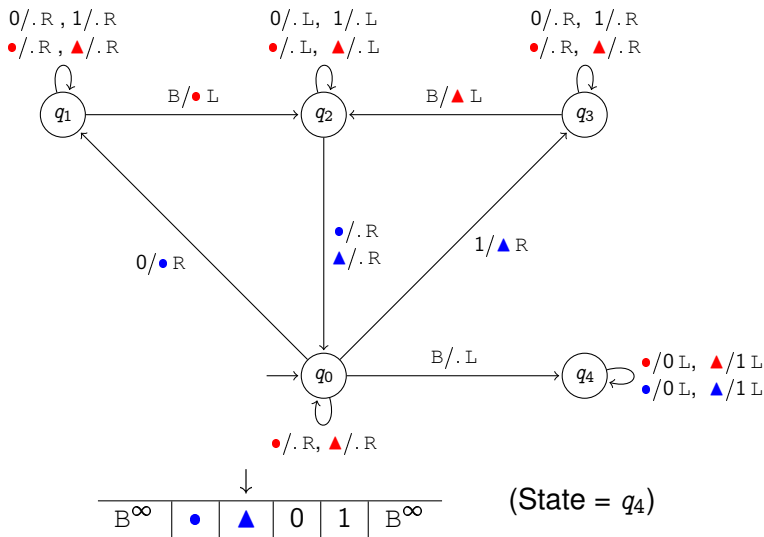
A TM that duplicates the input string $w \in \{0, 1\}^*$.



From Last Lecture: Example 2



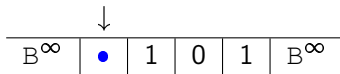
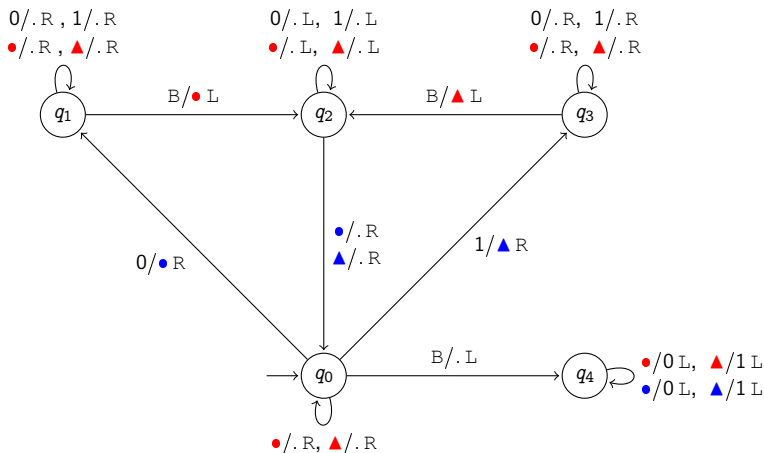
A TM that duplicates the input string $w \in \{0, 1\}^*$.



From Last Lecture: Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.

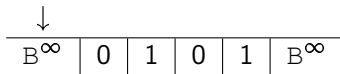
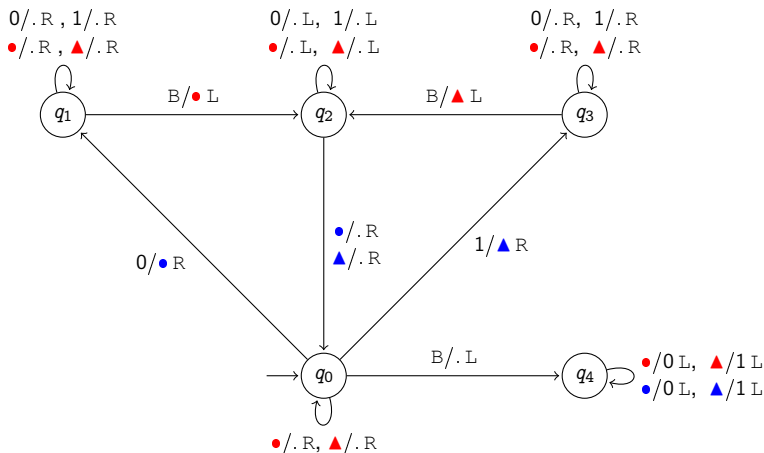


(State = q_4)

From Last Lecture: Example 2



A TM that duplicates the input string $w \in \{0, 1\}^*$.



(State = q_4)



The set $L(M)$ can be defined in two different ways.

1. A TM M **accepts by termination** the language of the input strings w for which it terminates:

$$L(M) = \{w \in \Sigma^* \mid q_0 w \vdash^* \perp\}$$

No need for accepting states.

Note: relation $\vdash^*$ above is the reflexive, transitive closure of $\vdash$.



The set $L(M)$ can be defined in two different ways.

1. A TM M **accepts by termination** the language of the input strings w for which it terminates:

$$L(M) = \{w \in \Sigma^* \mid q_0 w \vdash^* \perp\}$$

No need for accepting states.

Note: relation $\vdash^*$ above is the reflexive, transitive closure of $\vdash$.

2. $L(M)$ can also be defined by **termination in an accepting state**, extending M with a set $F \subseteq Q$:

$$L(M) = \{w \in \Sigma^* \mid \exists q_f \in F, u, v \in \Gamma^* : q_0 w \vdash^* u q_f v \vdash \perp\}$$



The set $L(M)$ can be defined in two different ways.

1. A TM M **accepts by termination** the language of the input strings w for which it terminates:

$$L(M) = \{w \in \Sigma^* \mid q_0 w \vdash^* \perp\}$$

No need for accepting states.

Note: relation $\vdash^*$ above is the reflexive, transitive closure of $\vdash$.

2. $L(M)$ can also be defined by **termination in an accepting state**, extending M with a set $F \subseteq Q$:

$$L(M) = \{w \in \Sigma^* \mid \exists q_f \in F, u, v \in \Gamma^* : q_0 w \vdash^* u q_f v \vdash \perp\}$$

This definition can be reduced to the first one by letting $F = Q$.

In fact, both definitions are equivalent.



A TM is **always terminating** if it terminates for every input.

Let L be a language.

- L is **semi-decidable** (or **recursively enumerable, RE**)
if there exists a TM M such that $L = L(M)$.
- L is **decidable** (or **recursive**)
if there is an always terminating TM that accepts L by termination in an accepting state.
- If L is decidable, then it is also semi-decidable.
The converse doesn't hold!

Outline



From Last Lecture

Variations of TMs

- Multitrack TMs

- The Example Revisited (I)

- Multitape TMs

- The Example Revisited (II)

- Simulating Multitape with Multitrack

Non-Deterministic TMs (NTMs)

- Adding Non-Determinism

- Complexity Classes

Closure Properties

Variations of TMs



- Extensions of TMs: multitrack, multitape, non-deterministic
- These generalized machines are convenient...



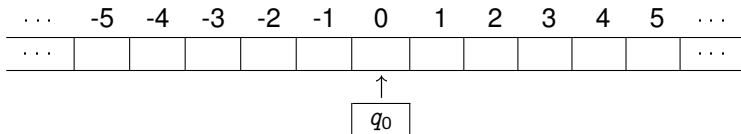
- Extensions of TMs: multitrack, multitape, non-deterministic
- These generalized machines are convenient...
- ...but don't add expressive power: the languages accepted by them are precisely those accepted by standard TMs



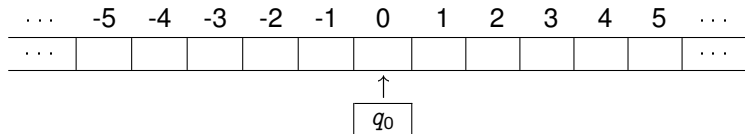
- Extensions of TMs: multitrack, multitape, non-deterministic
- These generalized machines are convenient...
- ...but don't add expressive power: the languages accepted by them are precisely those accepted by standard TMs

The extensions will be useful next lecture, when discussing **universal** Turing machines.

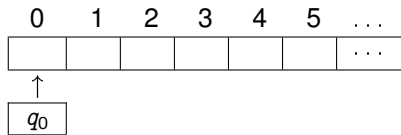
- The TMs seen in the previous lecture are already an extension: **two-way** TMs, for which the tape extends in both directions:



- The TMs seen in the previous lecture are already an extension: **two-way** TMs, for which the tape extends in both directions:



- But this is actually an extension of a **simple** TM in which there is a left boundary: the tape extends indefinitely only in one direction:



- A simple TM can simulate the actions of a two-way TM. This can be proved by using a TM with **two tracks**.

Multitrack Turing Machines (TMs)



- A TM in which the tape is divided into tracks
- A tape position in an n -track tape contains n symbols from the tape alphabet. The TM reads an entire tape position.

Multitrack Turing Machines (TMs)



- A TM in which the tape is divided into tracks
- A tape position in an n -track tape contains n symbols from the tape alphabet. The TM reads an entire tape position.
- In the case of a two-track TM, we would have:

Track 2	...	1	2	3	4	5	6	7	...
Track 1	...	a	b	c	d	e	f	g	...

↑
 q_i

The machine simultaneously reads b and 2.

Multitrack Turing Machines (TMs)



- A TM in which the tape is divided into tracks
- A tape position in an n -track tape contains n symbols from the tape alphabet. The TM reads an entire tape position.
- In the case of a two-track TM, we would have:

Track 2	...	1	2	3	4	5	6	7	...
Track 1	...	a	b	c	d	e	f	g	...

↑
 q_i

The machine simultaneously reads b and 2.

- A multitrack TM can be represented as a one-track TM using tuples. In the case of two-track TMs, ordered pairs suffice:

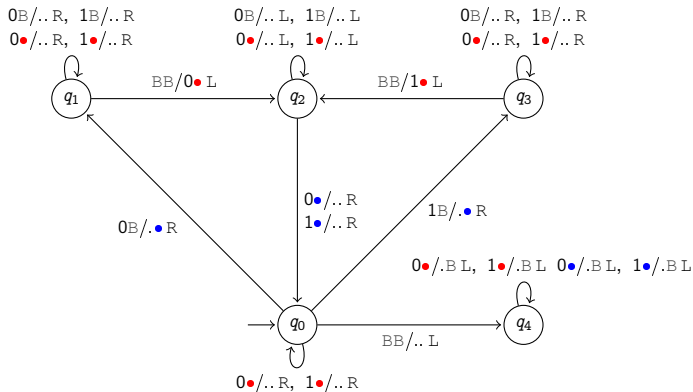
...	$(a, 1)$	$(b, 2)$	$(c, 3)$	$(d, 4)$	$(e, 5)$	$(f, 6)$	$(g, 7)$	...
-----	----------	----------	----------	----------	----------	----------	----------	-----

↑
 q_i

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.

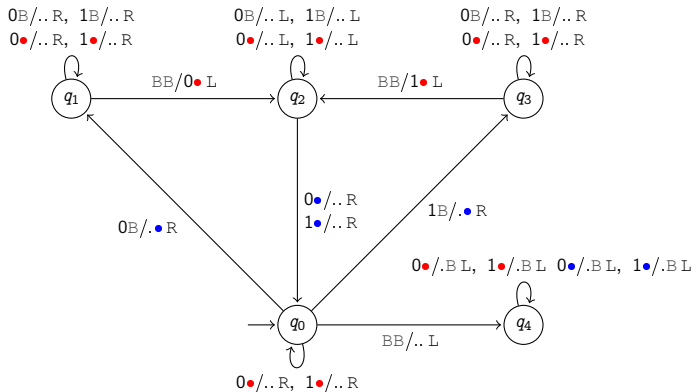


q_0						
↓						
B^∞	B	0	1	B	B	B^∞
B^∞	B	B	B	B	B	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.

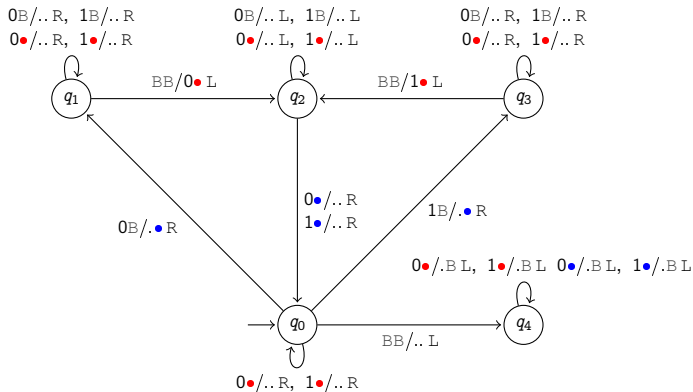


q_1						
B^∞	B	0	1	B	B	B^∞
B^∞	B	$\bullet$	B	B	B	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



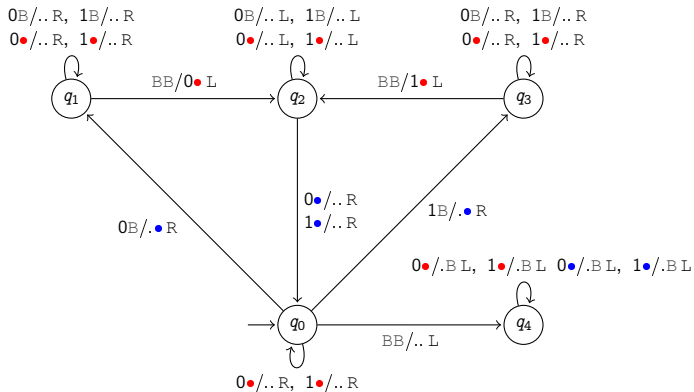
q_1

B^∞	B	0	1	B	B	B^∞
B^∞	B	•	B	B	B	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.

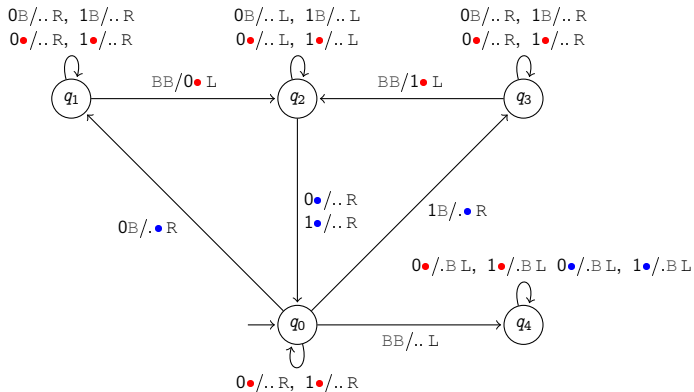


q_2						
B^∞	B	0	1	0	B	B^∞
B^∞	B	•	B	•	B	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.

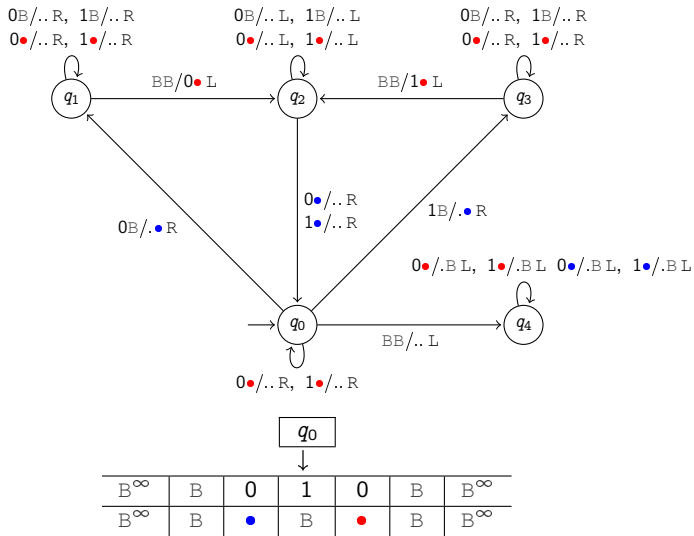


q_2						
↓						
B^∞	B	0	1	0	B	B^∞
B^∞	B	•	B	•	B	B^∞

Example 2



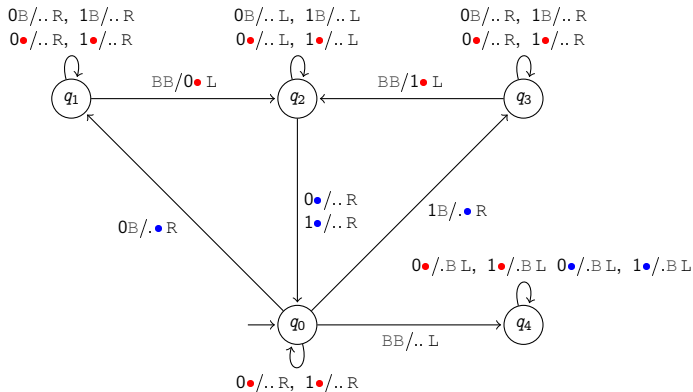
A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



q_3

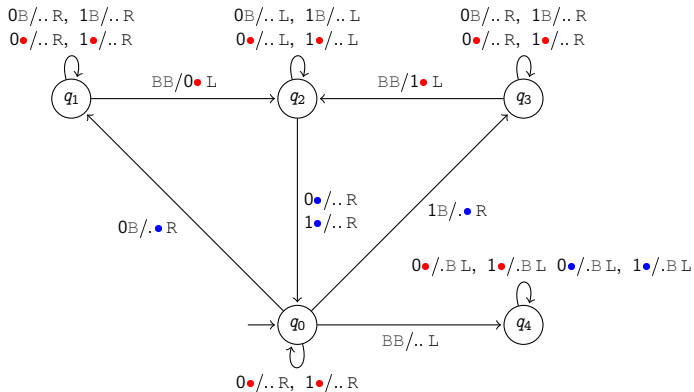
↓

B^∞	B	0	1	0	B	B^∞
B^∞	B	•	•	•	B	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



q_3

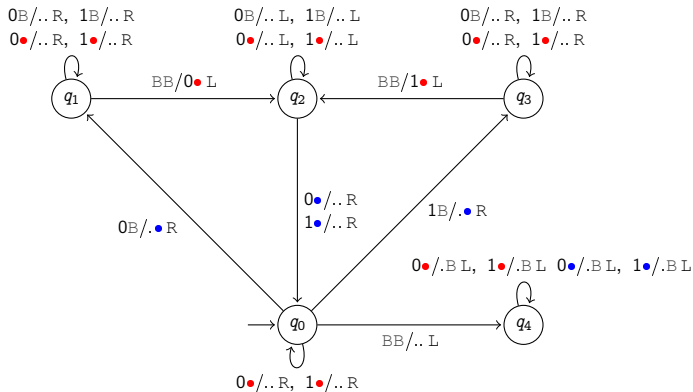
↓

B^∞	B	0	1	0	B	B^∞
B^∞	B	•	•	•	B	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



q₂

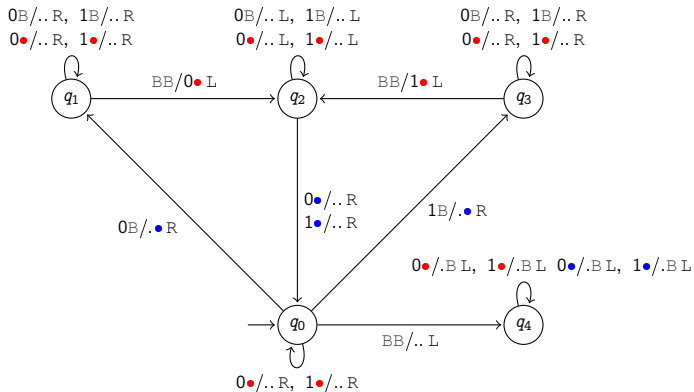
↓

B^∞	B	0	1	0	1	B^∞
B^∞	B	•	•	•	•	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.

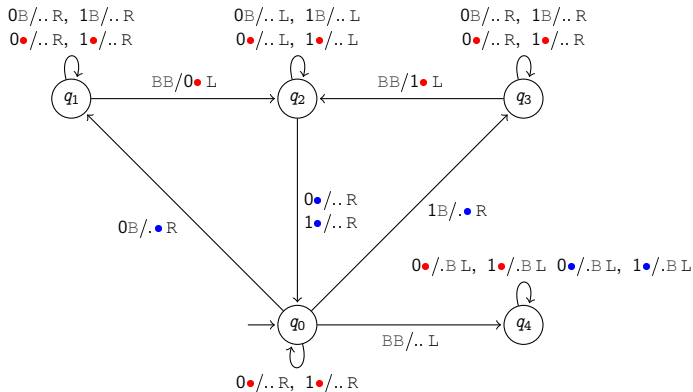


q_2						
↓						
B^∞	B	0	1	0	1	B^∞
B^∞	B	•	•	•	•	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



q_0

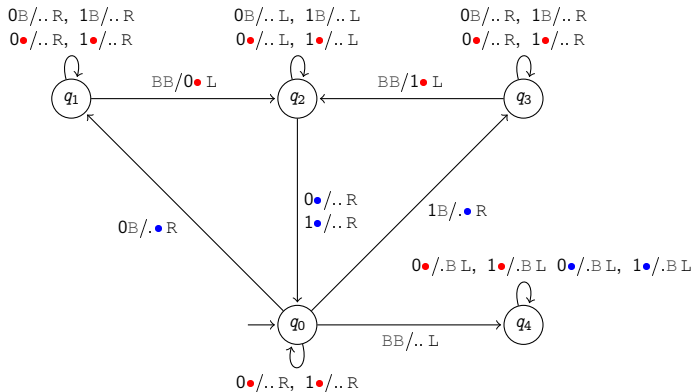
↓

B^∞	B	0	1	0	1	B^∞
B^∞	B	•	•	•	•	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



q_0

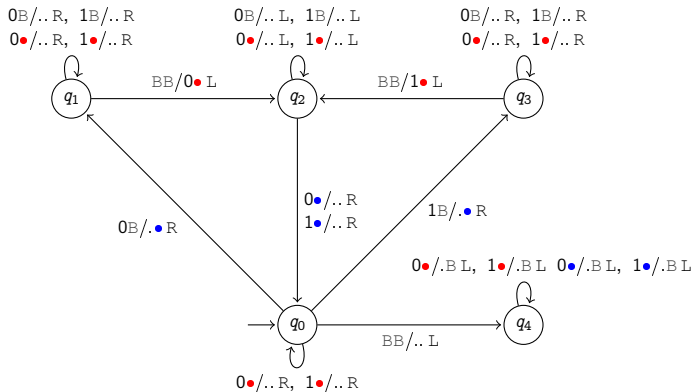
↓

B^∞	B	0	1	0	1	B^∞
B^∞	B	•	•	•	•	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



q_0

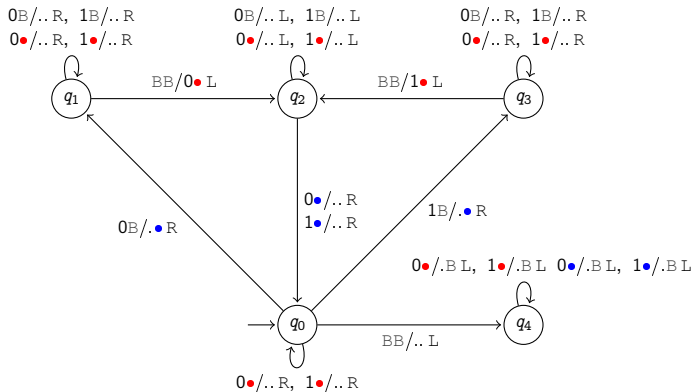
↓

B^∞	B	0	1	0	1	B^∞
B^∞	B	•	•	•	•	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



q_4

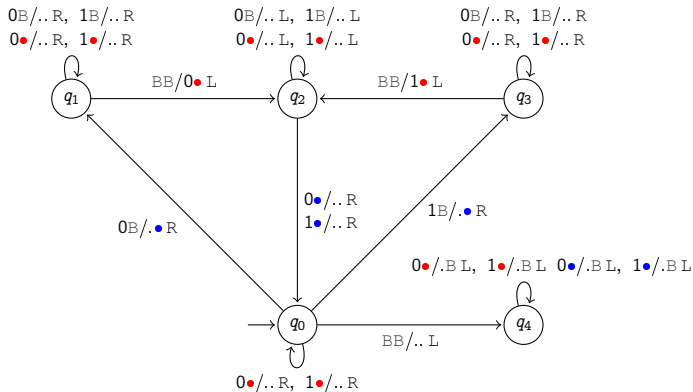
↓

B^∞	B	0	1	0	1	B^∞
B^∞	B	•	•	•	•	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



q_4

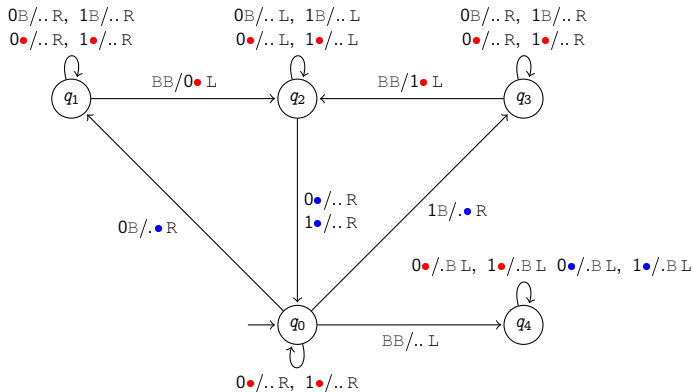
↓

B^∞	B	0	1	0	1	B^∞
B^∞	B	$\bullet$	$\bullet$	$\bullet$	B	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



q_4

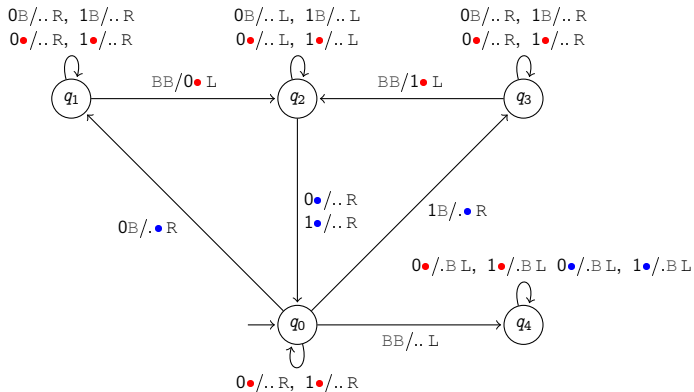
↓

B^∞	B	0	1	0	1	B^∞
B^∞	B	•	•	B	B	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



q_4

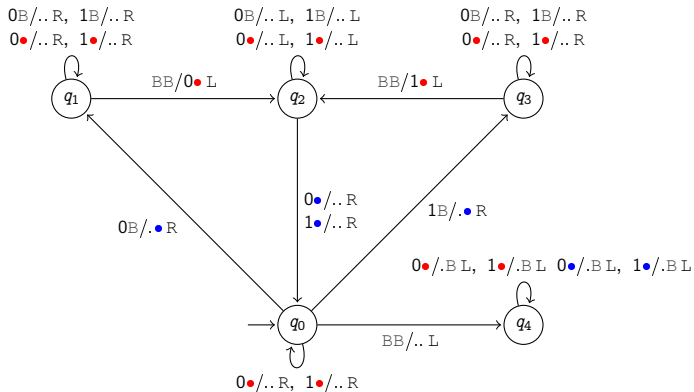
↓

B^∞	B	0	1	0	1	B^∞
B^∞	B	•	B	B	B	B^∞

Example 2



A multitrack TM that duplicates the input string $w \in \{0, 1\}^*$.



q_4

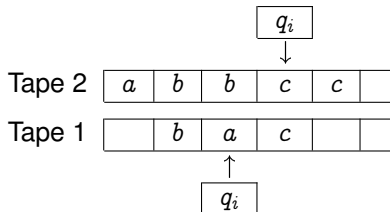
↓

B^∞	B	0	1	0	1	B^∞
B^∞	B	B	B	B	B	B^∞

Multitape TMs



- A k -tape TM consists of k tapes and k independent tape heads
- The TM reads the tapes simultaneously, but has only one state
- A two tape machine:



- A transition of a two-tape machine:

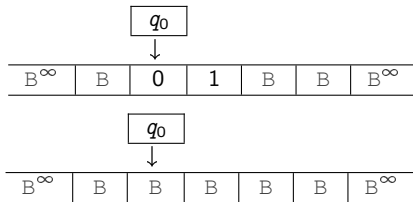
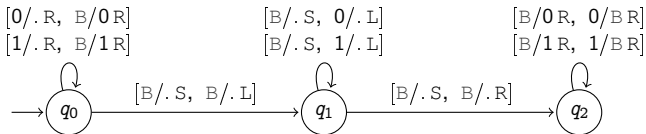
$$\delta(q_i, x_1, x_2) = [q_j; y_1, d_1; y_2, d_2]$$

- x_i and y_i are the old and new symbols on tape i ;
- q_i and q_j are the old and new states;
- $d_i \in \{L, R, S\}$ is the direction of movement for head i (S means “stationary”)

Example 2



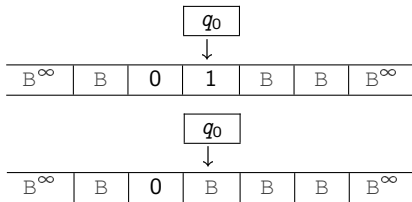
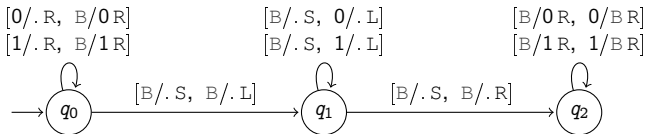
A multitape TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



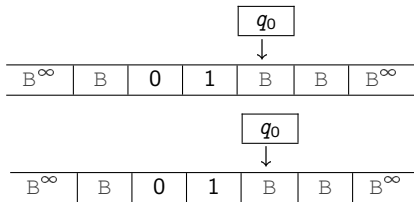
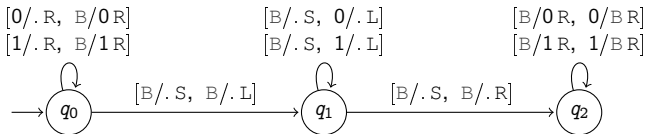
A multitape TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



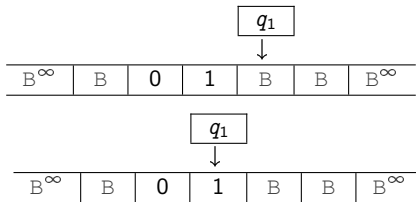
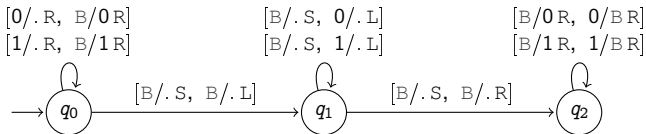
A multitape TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



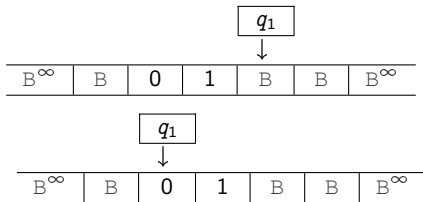
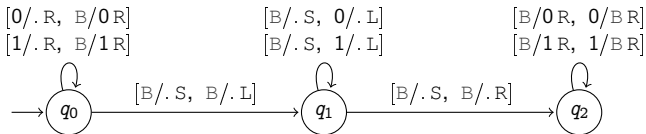
A multitape TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



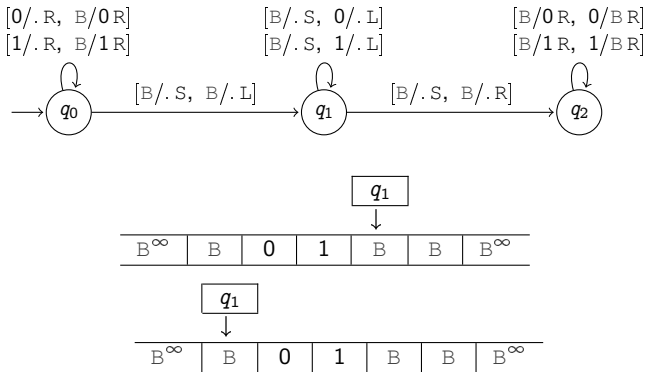
A multitape TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



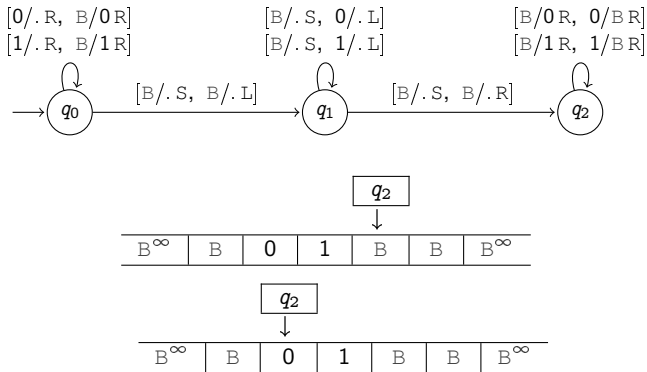
A multitape TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



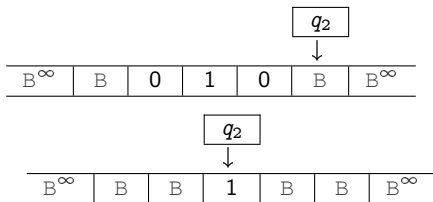
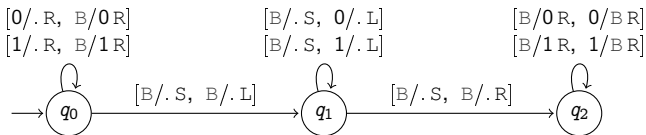
A multitape TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



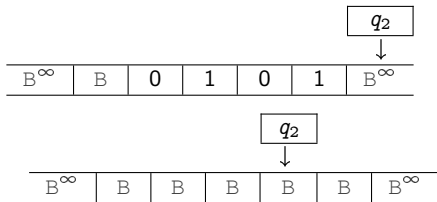
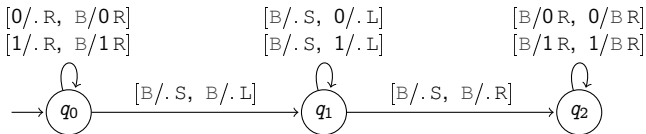
A multitape TM that duplicates the input string $w \in \{0, 1\}^*$.



Example 2



A multitape TM that duplicates the input string $w \in \{0, 1\}^*$.



Simulating Multitape with Multitrack

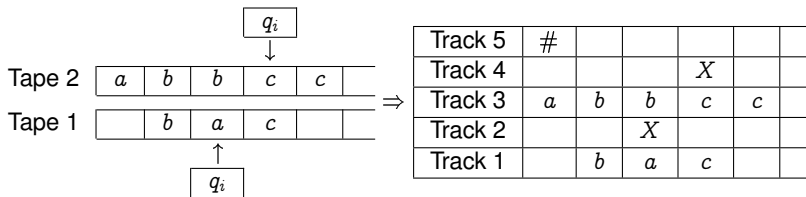


- Simulating a **two-tape** machine using a **five-track** machine:
 - ▶ Tracks 1 and 3 maintain the information on tapes 1 and 2
 - ▶ Tracks 2 and 4 use a symbol X to indicate the position of the heads
 - ▶ Track 5 uses a symbol $\#$ to control the simulation

Simulating Multitape with Multitrack



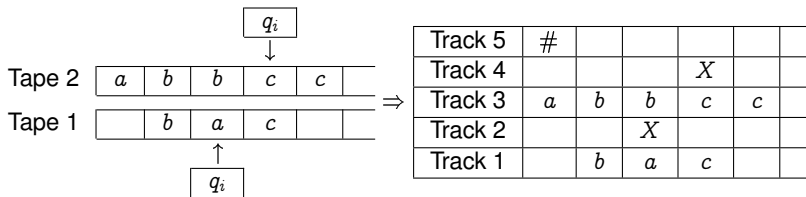
- Simulating a **two-tape** machine using a **five-track** machine:
 - ▶ Tracks 1 and 3 maintain the information on tapes 1 and 2
 - ▶ Tracks 2 and 4 use a symbol X to indicate the position of the heads
 - ▶ Track 5 uses a symbol $\#$ to control the simulation
- Graphically:



Simulating Multitape with Multitrack

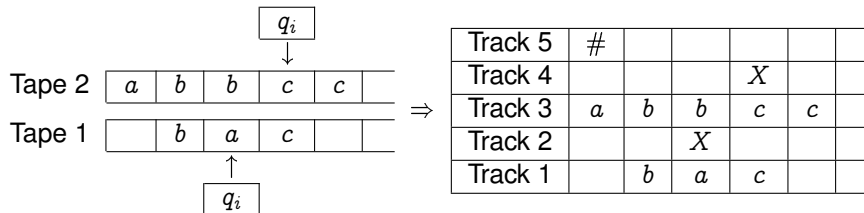


- Simulating a **two-tape** machine using a **five-track** machine:
 - ▶ Tracks 1 and 3 maintain the information on tapes 1 and 2
 - ▶ Tracks 2 and 4 use a symbol X to indicate the position of the heads
 - ▶ Track 5 uses a symbol $\#$ to control the simulation
- Graphically:



- In general, a language accepted by a k -tape machine is accepted by a $2k + 1$ -track machine

Simulating Multitape with Multitrack



Consider a (multitape) transition $\delta(q_i, x_1, x_2) = [q_j; y_1, d_1; y_2, d_2]$.

Its simulation in the multitrack machine roughly involves:

1. Finding the x_1 and x_2 in T1 and T3, using the X s in T2 and T4.
2. With x_1 and x_2 , the y_1 and y_2 to be printed and the directions d_1 and d_2 can be determined.
3. Printing y_1 and y_2 in T1 and T3, and moving the X s in T2 and T4, according to d_1 and d_2 .

Outline



From Last Lecture

Variations of TMs

- Multitrack TMs

- The Example Revisited (I)

- Multitape TMs

- The Example Revisited (II)

- Simulating Multitape with Multitrack

Non-Deterministic TMs (NTMs)

- Adding Non-Determinism

- Complexity Classes

Closure Properties

Non-Deterministic TMs (NTMs)



- Just as the other machines, TMs can be non-deterministic
- This means that the transition function is defined as

$$\delta : Q \times \Gamma \rightarrow$$

Non-Deterministic TMs (NTMs)



- Just as the other machines, TMs can be non-deterministic
- This means that the transition function is defined as

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

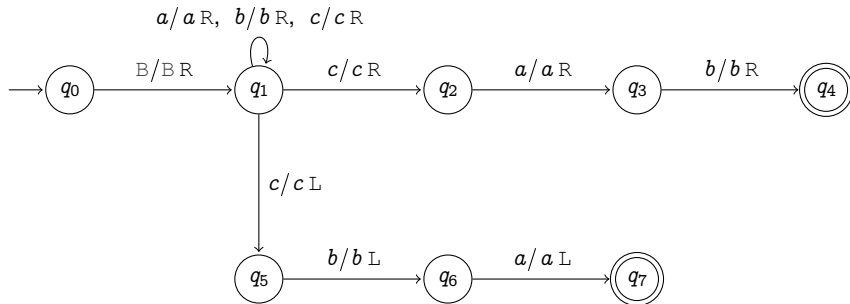


- Just as the other machines, TMs can be non-deterministic
- This means that the transition function is defined as

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{\text{L}, \text{R}\})$$

- When more than one transition is possible, the computation arbitrarily chooses one
- An NTM may produce several computations for a single input string. The string is accepted if there is a computation that terminates.

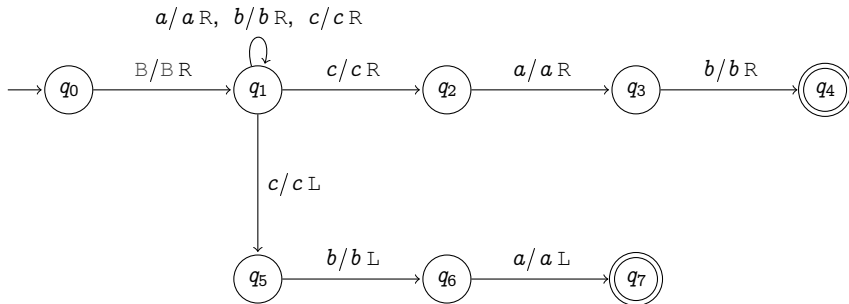
Example 3: An NTM



Example 3: An NTM



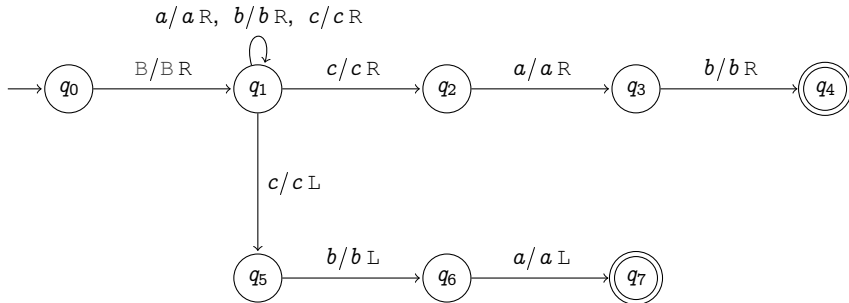
Accepting strings containing an occurrence of c that is preceded **or** followed by ab :



Example 3: An NTM

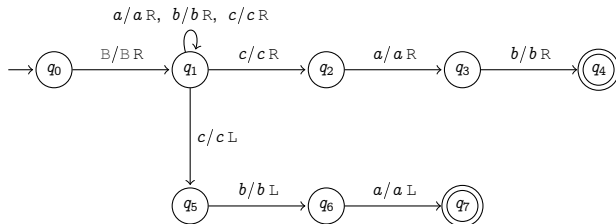


Accepting strings containing an occurrence of c that is preceded **or** followed by ab :



Thanks to non-determinism, the computation chooses a c and then chooses one of the conditions to check.

Example 3: An NTM



Three different computations for the same input string $acab$:

$\rightarrow [q_0] B acab B$

$\vdash$

$\vdash$

$\vdash$

$\vdash$

$\vdash$

$\rightarrow [q_0] B acab B$

$\vdash$

$\vdash$

$\vdash$

$\vdash$

$\vdash$

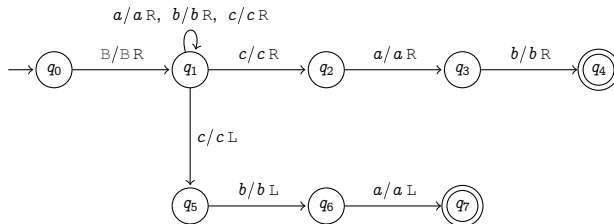
$\rightarrow [q_0] B acab B$

$\vdash$

$\vdash$

$\vdash$

Example 3: An NTM



Three different computations for the same input string $acab$:

$\rightarrow [q_0]BacabB$

$\vdash B[q_1]acabB$

$\vdash Ba[q_1]cabB$

$\vdash Bac[q_1]abB$

$\vdash Baca[q_1]bB$

$\vdash Bacab[q_1]B$

$\rightarrow [q_0]BacabB$

$\vdash$

$\vdash$

$\vdash$

$\vdash$

$\vdash$

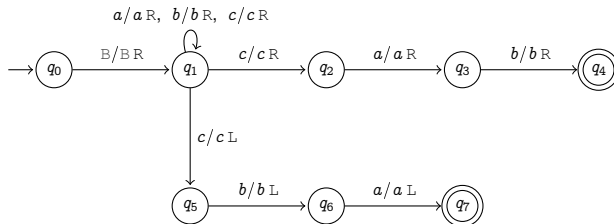
$\rightarrow [q_0]BacabB$

$\vdash$

$\vdash$

$\vdash$

Example 3: An NTM



Three different computations for the same input string $acab$:

$\rightarrow [q_0]_B acab_B$

$\vdash_B [q_1]_B acab_B$

$\vdash_B a[q_1]_B cab_B$

$\vdash_B ac[q_1]_B ab_B$

$\vdash_B aca[q_1]_B b_B$

$\vdash_B acab[q_1]_B$

$\rightarrow [q_0]_B acab_B$

$\vdash_B [q_1]_B acab_B$

$\vdash_B a[q_1]_B cab_B$

$\vdash_B ac[q_2]_B ab_B$

$\vdash_B aca[q_3]_B b_B$

$\vdash_B acab[q_4]_B$

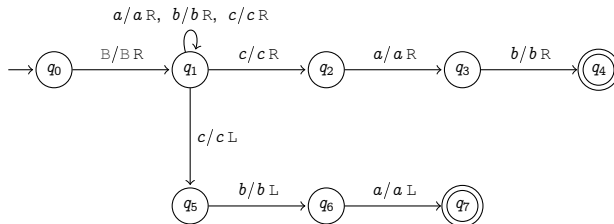
$\rightarrow [q_0]_B acab_B$

$\vdash$

$\vdash$

$\vdash$

Example 3: An NTM



Three different computations for the same input string $acab$:

$\rightarrow [q_0]_B acab_B$

$\vdash_B [q_1]_B acab_B$

$\vdash_B a[q_1]_B cab_B$

$\vdash_B ac[q_1]_B ab_B$

$\vdash_B aca[q_1]_B b_B$

$\vdash_B acab[q_1]_B$

$\rightarrow [q_0]_B acab_B$

$\vdash_B [q_1]_B acab_B$

$\vdash_B a[q_1]_B cab_B$

$\vdash_B ac[q_2]_B ab_B$

$\vdash_B aca[q_3]_B b_B$

$\vdash_B acab[q_4]_B$

$\rightarrow [q_0]_B acab_B$

$\vdash_B [q_1]_B acab_B$

$\vdash_B a[q_1]_B cab_B$

$\vdash_B [q_5]_B acab_B$

Non-Deterministic TMs (NTMs)



- Just as other machines we have seen, TMs can be non-deterministic
- This means that the transition function is defined as

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{\mathbb{L}, R\})$$

- When more than one transition is possible, the computation arbitrarily chooses one
- An NTM may produce several computations for a single input string
- The reader gives a procedure to represent NTM computations using a (deterministic) two-tape TM (Theorem 5.4)
- Non-determinism + multitracks + multitape?
Combinations are possible and handled as expected



The following are equivalent:

- Simple TMs
- Two-way TMs
- Multitrack TMs
- Multitape TMs
- Non-deterministic TMs (NTMs)
- Non-deterministic, multitrack TMs
- Non-deterministic, multitape TMs



Given an NTM with a set of accepting states, there are three kinds of computations:

1. Terminating and accepting
2. Terminating and non-accepting
3. Non terminating (infinite!)

An input is accepted iff it has at least one accepting computation (it may also have non-accepting and non-terminating computations)

A TM is **always terminating** if for every input string every computation terminates



A TM is **always terminating** if it terminates for every input.

Let L be a language.

- L is **semi-decidable** (or **recursively enumerable, RE**)
if there exists a TM M such that $L = L(M)$.
- L is **decidable** (or **recursive**)
if there is an always terminating TM that accepts L by termination in an accepting state.
- If L is decidable, then it is also semi-decidable.
The converse doesn't hold!



A (non)deterministic TM M has **time complexity** $T(n)$ if M is guaranteed to terminate in at most $T(n)$ steps for every input string w of length n (regardless of whether w is accepted).

Let L be a language and let $T(n)$ be a **polynomial function**:

- L belongs to the class $\mathcal{P}$ if there is a deterministic TM M with $L = L(M)$ and with time complexity $T(n)$.
- L belongs to the class $\mathcal{NP}$ if there is an NTM M with $L = L(M)$ and with time complexity $T(n)$.
- Because every deterministic TM can be regarded as an NTM with the same time complexity, we have $\mathcal{P} \subseteq \mathcal{NP}$.
- Conjecture: $\mathcal{P} \neq \mathcal{NP}$.



- Everything that can be computed with a DTM, can be computed with an ordinary computer at least with the same efficiency, up-to memory extensions.
- Everything that can be computed with such an extendable computer, say in n steps, can be computed on a deterministic Turing machine in $T(n)$ steps for some polynomial $T(n)$.
- Ordinary computers are closer to the DTM than to the NTM.

Outline



From Last Lecture

Variations of TMs

- Multitrack TMs

- The Example Revisited (I)

- Multitape TMs

- The Example Revisited (II)

- Simulating Multitape with Multitrack

Non-Deterministic TMs (NTMs)

- Adding Non-Determinism

- Complexity Classes

Closure Properties

Closure Properties



We know:

$$L \text{ is decidable} \Rightarrow L \text{ is semi-decidable} \quad (*)$$

Furthermore:

1. L is decidable $\Rightarrow \bar{L}$ is decidable
2. L and $\bar{L}$ are semi-decidable $\Leftrightarrow L$ is decidable
3. L is semi-decidable $\Leftrightarrow L^*$ is semi-decidable
4. L_1 and L_2 are semi-decidable $\Rightarrow L_1 L_2, L_1 \cup L_2,$ and $L_1 \cap L_2$ are semi-decidable

Key ideas:

1. Use the complement of the set of accepting states.
2. $\Rightarrow$) Given M_1 and M_2 for L and $\bar{L}$, devise a two-tape TM that runs M_1 and M_2 in lockstep.
 $\Leftarrow$) Immediate from (1) and (*)
3. Exercise 5.13
4. These properties proven by building appropriate TMs.



This lecture:

- ▶ Variants of Turing machines
- ▶ DTMs, NTMs, and their complexity classes
- ▶ Closure properties

Next Lecture: Thursday, May 28

- Decision problems, in particular the halting problem
- Problems, languages, and (semi-)decidability
- Universal Turing machines